

DATABASE SYSTEMS

OPEN-ENDED LAB REPORT

Car Rental Management System

Student Name	Fatima Arshad
Roll No.	2024-SE-24
Subject	Database Systems
Lab Title	DBMS Open-Ended Lab — Car Rental Management System
Submitted By	Sir Awais

1. CLOs Assessed

CLO-6 (C-4): Analyze relational databases to identify inefficiencies and optimize their design using normalization, JOINS, views, triggers, and stored procedures.

CLO-7 (C-5): Design and develop a complete database solution for a real-world problem.

2. Problem Scenario

CarGo Rentals is a local car rental company that rents vehicles to customers for daily or weekly use. The company currently maintains customer, vehicle, rental, return, and payment information using spreadsheets. As the number of rentals increases, duplicate information, inconsistent records, and reporting difficulties have become major problems.

The proposed relational database manages customers, vehicles, rentals, returns, and payments. It calculates rental charges, prevents a vehicle from being rented while an active rental exists, and supports management reports for customer rental history, currently rented vehicles, and vehicle rental activity.

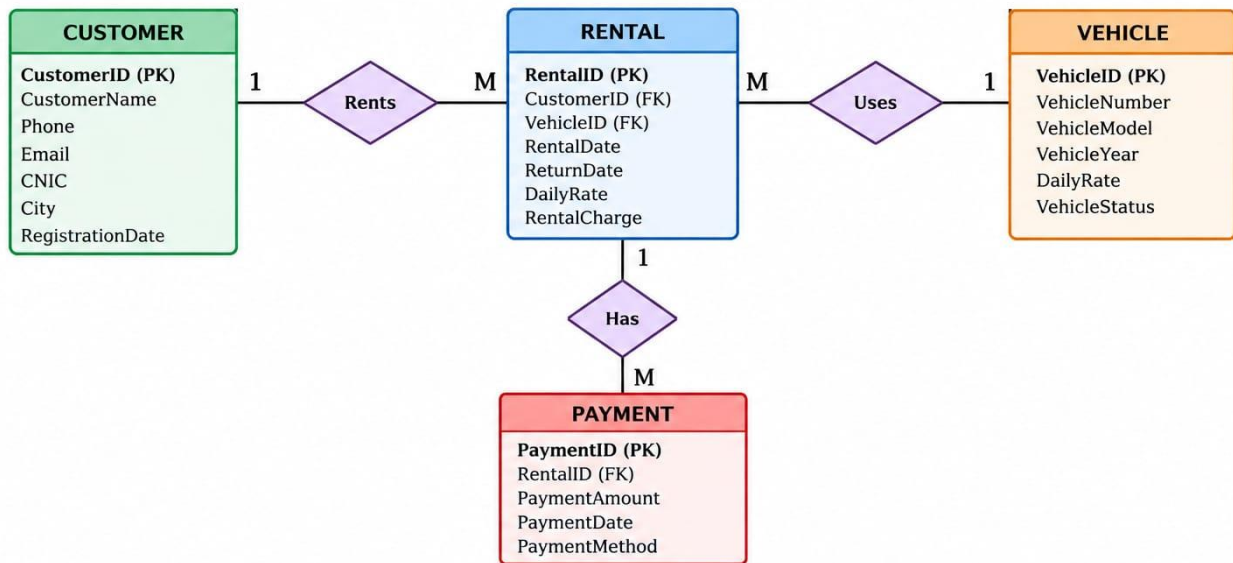
3. Task 1 — Database Design and Implementation

3.1 Proposed Tables

Table	Purpose	Primary Key
Customer	Stores customer information.	CustomerID
Vehicle	Stores vehicle information.	VehicleID
Rental	Stores rental transactions.	RentalID
Payment	Stores payment information.	PaymentID

3.2 Relationship Design

4.2 Relationship Design



Relationship Explanation

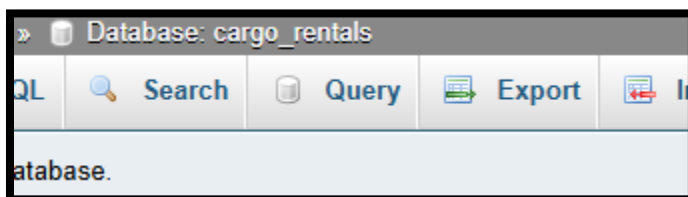
1. One customer can have many rentals.
2. One vehicle can appear in many rental records over time.
3. Each rental belongs to one customer.
4. Each rental belongs to one vehicle.
5. Each rental can have one payment record in the proposed design.

- One customer can have many rentals.
- One vehicle can appear in many rental records over time.
- Each rental belongs to one customer.
- Each rental belongs to one vehicle.
- Each rental can have one payment record in the proposed design.

The relationships can be represented as: **CUSTOMER 1:M RENTAL**, **VEHICLE 1:M RENTAL** over time, and **RENTAL 1:0..1 PAYMENT**.

3.3 Database Creation

```
CREATE DATABASE IF NOT EXISTS cargo_rentals;  
USE cargo_rentals;
```



3.4 Reset Tables

```
DROP TABLE IF EXISTS Payment;  
DROP TABLE IF EXISTS Rental;  
DROP TABLE IF EXISTS Vehicle;  
DROP TABLE IF EXISTS Customer;
```

3.5 Customer Table

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY AUTO_INCREMENT,  
    CustomerName VARCHAR(100) NOT NULL,  
    Phone VARCHAR(20) NOT NULL UNIQUE,  
    Email VARCHAR(100) UNIQUE,  
    CNIC VARCHAR(20) UNIQUE,  
    City VARCHAR(50) NOT NULL,  
    RegistrationDate DATE NOT NULL DEFAULT (CURRENT_DATE)  
);
```

CustomerID uniquely identifies each customer. Phone, Email, and CNIC are protected from duplicates where provided, while important fields are NOT NULL and the registration date has a default value.

3.6 Vehicle Table

```
CREATE TABLE Vehicle (  
    VehicleID INT PRIMARY KEY AUTO_INCREMENT,  
    VehicleNumber VARCHAR(20) NOT NULL UNIQUE,  
    VehicleModel VARCHAR(50) NOT NULL,  
    VehicleYear INT NOT NULL,  
    DailyRate DECIMAL(10,2) NOT NULL,  
    VehicleStatus VARCHAR(20) NOT NULL DEFAULT 'Available',  
    CHECK (DailyRate > 0),  
    CHECK (VehicleYear BETWEEN 2000 AND 2030),  
    CHECK (VehicleStatus IN ('Available', 'Maintenance'))  
);
```

3.7 Rental Table

```
CREATE TABLE Rental (  
    RentalID INT PRIMARY KEY AUTO_INCREMENT,  
    CustomerID INT NOT NULL,  
    VehicleID INT NOT NULL,  
    RentalDate DATE NOT NULL,  
    ReturnDate DATE NULL,  
    DailyRate DECIMAL(10,2) NOT NULL,  
    RentalCharge DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
```

```

FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID),
CHECK (DailyRate > 0),
CHECK (RentalCharge >= 0),
CHECK (ReturnDate IS NULL OR ReturnDate >= RentalDate)
);

```

3.8 Payment Table

```

CREATE TABLE Payment (
    PaymentID INT PRIMARY KEY AUTO_INCREMENT,
    RentalID INT NOT NULL UNIQUE,
    PaymentAmount DECIMAL(10,2) NOT NULL,
    PaymentDate DATE NOT NULL DEFAULT (CURRENT_DATE),
    PaymentMethod VARCHAR(20) NOT NULL DEFAULT 'Cash',
    FOREIGN KEY (RentalID) REFERENCES Rental(RentalID),
    CHECK (PaymentAmount >= 0),
    CHECK (PaymentMethod IN ('Cash', 'Card', 'Bank Transfer', 'Online'))
);

```

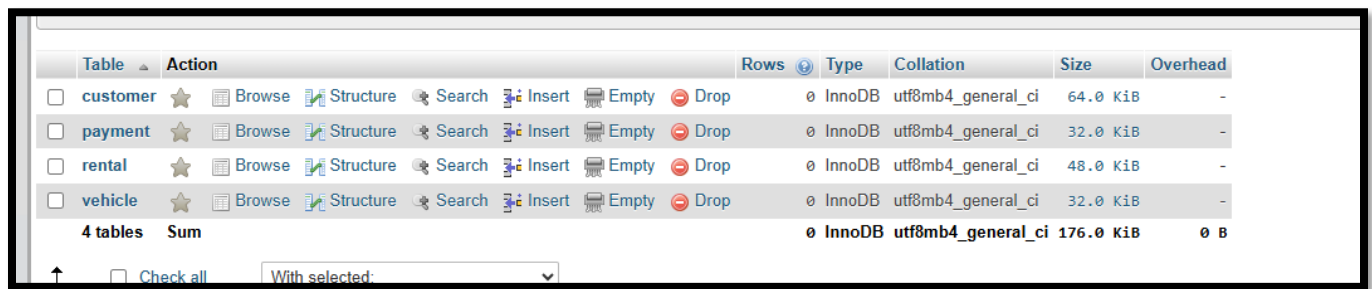


Table	Action	Rows	Type	Collation	Size	Overhead
customer	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	64.0 KiB	-
payment	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
rental	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
vehicle	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
4 tables	Sum	0	InnoDB	utf8mb4_general_ci	176.0 KiB	0 B

3.9 Sample Data

○ Customer Data

```

INSERT INTO Customer
(CustomerName, Phone, Email, CNIC, City)
VALUES
('Ali Khan','0300-1234567','ali.khan@gmail.com','37405-1234567-1','Lahore'),
('Sara Ahmed','0311-2345678','sara.ahmed@gmail.com','35202-2345678-2','Islamabad'),
('Hamza Raza','0322-3456789','hamza.raza@gmail.com','61101-3456789-3','Karachi'),
('Ayesha Noor','0333-4567890','ayesha.noor@gmail.com','37405-4567890-4','Lahore'),
('Bilal Ahmed','0344-5678901','bilal.ahmed@gmail.com','35202-5678901-5','Rawalpindi'),
('Fatima Ali','0355-6789012','fatima.ali@gmail.com','61101-6789012-6','Multan'),
('Usman Tariq','0366-7890123','usman.tariq@gmail.com','37405-7890123-7','Peshawar');

SELECT * FROM Customer;

```

`SELECT * FROM `customer``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

	CustomerID	CustomerName	Phone	Email	CNIC	City	RegistrationDate
☐ Edit Copy Delete	1	Ali Khan	0300-1234567	ali.khan@gmail.com	37405-1234567-1	Lahore	2026-09-19
☐ Edit Copy Delete	2	Sara Ahmed	0311-2345678	sara.ahmed@gmail.com	35202-2345678-2	Islamabad	2026-09-19
☐ Edit Copy Delete	3	Hamza Raza	0322-3456789	hamza.raza@gmail.com	61101-3456789-3	Karachi	2026-09-19
☐ Edit Copy Delete	4	Ayesha Noor	0333-4567890	ayesha.noor@gmail.com	37405-4567890-4	Lahore	2026-09-19
☐ Edit Copy Delete	5	Bilal Ahmed	0344-5678901	bilal.ahmed@gmail.com	35202-5678901-5	Rawalpindi	2026-09-19
☐ Edit Copy Delete	6	Fatima Ali	0355-6789012	fatima.ali@gmail.com	61101-6789012-6	Multan	2026-09-19
☐ Edit Copy Delete	7	Usman Tariq	0366-7890123	usman.tariq@gmail.com	37405-7890123-7	Peshawar	2026-09-19

○ Vehicle Data

INSERT INTO Vehicle

(VehicleNumber, VehicleModel, VehicleYear, DailyRate, VehicleStatus)

VALUES

('ABC-123','Toyota Corolla',2022,5000.00,'Available'),

('XYZ-456','Honda Civic',2023,6500.00,'Available'),

('LEA-789','Suzuki Swift',2021,4000.00,'Available'),

('ISB-321','Toyota Yaris',2022,4500.00,'Available'),

('KHI-654','Honda City',2023,5500.00,'Available'),

('RWP-987','Suzuki Wagon R',2020,3500.00,'Maintenance'),

('PSH-555','Toyota Fortuner',2024,12000.00,'Available');

SELECT * FROM Vehicle;

`SELECT * FROM `vehicle``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: Filter rows: Sort by key:

Extra options

		VehicleID	VehicleNumber	VehicleModel	VehicleYear	DailyRate	VehicleStatus
☐	Edit Copy Delete	1	ABC-123	Toyota Corolla	2022	5000.00	Available
☐	Edit Copy Delete	2	XYZ-456	Honda Civic	2023	6500.00	Available
☐	Edit Copy Delete	3	LEA-789	Suzuki Swift	2021	4000.00	Available
☐	Edit Copy Delete	4	ISB-321	Toyota Yaris	2022	4500.00	Available
☐	Edit Copy Delete	5	KHI-654	Honda City	2023	5500.00	Available
☐	Edit Copy Delete	6	RWP-987	Suzuki Wagon R	2020	3500.00	Maintenance
☐	Edit Copy Delete	7	PSH-555	Toyota Fortuner	2024	12000.00	Available

○ Rental Data

INSERT INTO Rental

(CustomerID, VehicleID, RentalDate, ReturnDate, DailyRate, RentalCharge)

VALUES

(1,1,'2026-09-01','2026-09-04',5000.00,15000.00),

(2,2,'2026-09-05','2026-09-08',6500.00,19500.00),

(3,3,'2026-09-10','2026-09-12',4000.00,8000.00),

(4,4,'2026-09-11',NULL,4500.00,0.00),

(5,5,'2026-09-12',NULL,5500.00,0.00);

SELECT * FROM Rental;

`SELECT * FROM `rental``

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		RentalID	CustomerID	VehicleID	RentalDate	ReturnDate	DailyRate	RentalCharge
☐	Edit Copy Delete	1	1	1	2026-09-01	2026-09-04	5000.00	15000.00
☐	Edit Copy Delete	2	2	2	2026-09-05	2026-09-08	6500.00	19500.00
☐	Edit Copy Delete	3	3	3	2026-09-10	2026-09-12	4000.00	8000.00
☐	Edit Copy Delete	4	4	4	2026-09-11	NULL	4500.00	0.00
☐	Edit Copy Delete	5	5	5	2026-09-12	NULL	5500.00	0.00

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

○ Payment Data

INSERT INTO Payment

(RentalID, PaymentAmount, PaymentMethod)

VALUES

(1,15000.00,'Cash'),

(2,19500.00,'Card'),

(3,8000.00,'Online');

SELECT * FROM Payment;

`SELECT * FROM `payment``

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		PaymentID	RentalID	PaymentAmount	PaymentDate	PaymentMethod
☐	Edit Copy Delete	1	1	15000.00	2026-09-19	Cash
☐	Edit Copy Delete	2	2	19500.00	2026-09-19	Card
☐	Edit Copy Delete	3	3	8000.00	2026-09-19	Online

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

4. Task 2 — Normalization

4.1 Original Structure

**RentalID | CustomerName | CustomerPhone | VehicleNumber |
VehicleModel | DailyRate | RentalDate | ReturnDate | PaymentAmount**

R001 | Ali Khan | 0300-1234567 | ABC-123 |
Toyota Corolla | 5000 | 2026-09-01 | 2026-09-04 | 15000

The single-record design repeats customer and vehicle information across rental records and can cause update, insertion, and deletion anomalies.

4.2 First Normal Form (1NF)

All attributes contain atomic values, there are no repeating groups, and each row represents one rental.

```
Rental(  
  RentalID,  
  CustomerName,  
  CustomerPhone,  
  VehicleNumber,  
  VehicleModel,  
  DailyRate,  
  RentalDate,  
  ReturnDate,  
  PaymentAmount  
)
```

4.3 Second Normal Form (2NF)

RentalID is a single-column primary key, so there is no partial dependency on a part of a composite key. The relation therefore satisfies the formal 2NF requirement after 1NF. The remaining issue is transitive dependency.

CustomerPhone -> CustomerName

VehicleNumber -> VehicleModel, DailyRate

RentalID -> CustomerPhone, VehicleNumber, RentalDate, ReturnDate, PaymentAmount

4.4 Third Normal Form (3NF)

The relation is decomposed into separate entities so that non-key attributes depend directly on their own table's key.

Customer (
 CustomerID, CustomerName, Phone, Email, CNIC, City, RegistrationDate

)

Vehicle (

VehicleID, VehicleNumber, VehicleModel, VehicleYear, DailyRate, VehicleStatus

)

Rental (

RentalID, CustomerID, VehicleID, RentalDate, ReturnDate, DailyRate, RentalCharge

)

Payment (

PaymentID, RentalID, PaymentAmount, PaymentDate, PaymentMethod

)

CustomerID -> CustomerName, Phone, Email, CNIC, City, RegistrationDate

VehicleID -> VehicleNumber, VehicleModel, VehicleYear, DailyRate, VehicleStatus

RentalID -> CustomerID, VehicleID, RentalDate, ReturnDate, DailyRate, RentalCharge

PaymentID -> RentalID, PaymentAmount, PaymentDate, PaymentMethod

This decomposition reduces redundancy and prevents common update, insertion, and deletion anomalies.

5. Task 3 — JOIN Queries

5.1 Query 1

Display the customer name, vehicle number, vehicle model, rental date, and return date for every rental.

- SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel,
r.RentalDate, r.ReturnDate
FROM Rental r
INNER JOIN Customer c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicle v ON r.VehicleID = v.VehicleID
ORDER BY r.RentalDate;

Showing rows 0 - 4 (5 total, Query took 0.0005 seconds.) [RentalDate: 2026-09-01... - 2026-09-12...]

```
SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate FROM Rental r INNER JOIN Customer c ON r.CustomerID = c.CustomerID INNER JOIN Vehicle v ON r.VehicleID = v.VehicleID ORDER BY r.RentalDate;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04
Sara Ahmed	XYZ-456	Honda Civic	2026-09-05	2026-09-08
Hamza Raza	LEA-789	Suzuki Swift	2026-09-10	2026-09-12
Ayesha Noor	ISB-321	Toyota Yaris	2026-09-11	NULL
Bilal Ahmed	KHI-654	Honda City	2026-09-12	NULL

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

INNER JOIN returns the matching customer and vehicle information for each rental.

5.2 Query 2

Display all customers and the vehicles they have rented, including customers who have never rented.

- SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel,
r.RentalDate, r.ReturnDate
FROM Customer c
LEFT JOIN Rental r ON c.CustomerID = r.CustomerID
LEFT JOIN Vehicle v ON r.VehicleID = v.VehicleID
ORDER BY c.CustomerName;

Showing rows 0 - 6 (7 total, Query took 0.0007 seconds.) [CustomerName: ALI KHAN... - USMAN TARIQ...]

```
SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate FROM Customer c LEFT JOIN Rental r ON c.CustomerID = r.CustomerID LEFT JOIN Vehicle v ON r.VehicleID = v.VehicleID ORDER BY c.CustomerName;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04
Ayesha Noor	ISB-321	Toyota Yaris	2026-09-11	NULL
Bilal Ahmed	KHI-654	Honda City	2026-09-12	NULL
Fatima Ali	NULL	NULL	NULL	NULL
Hamza Raza	LEA-789	Suzuki Swift	2026-09-10	2026-09-12
Sara Ahmed	XYZ-456	Honda Civic	2026-09-05	2026-09-08
Usman Tariq	NULL	NULL	NULL	NULL

Activate Windows

LEFT JOIN keeps customers with zero rentals in the result.

5.3 Query 3

Display all vehicles and their current rental information, including vehicles that are not currently rented.

- `SELECT v.VehicleNumber, v.VehicleModel, v.VehicleStatus,
c.CustomerName, r.RentalDate, r.ReturnDate
FROM Vehicle v
LEFT JOIN Rental r
ON v.VehicleID = r.VehicleID
AND r.ReturnDate IS NULL
LEFT JOIN Customer c ON r.CustomerID = c.CustomerID
ORDER BY v.VehicleNumber;`

✓ Showing rows 0 - 6 (7 total, Query took 0.0006 seconds.) [VehicleNumber: ABC-123... - XYZ-456...]

```
SELECT v.VehicleNumber, v.VehicleModel, v.VehicleStatus, c.CustomerName, r.RentalDate, r.ReturnDate FROM Vehicle v LEFT JOIN Rental r ON v.VehicleID = r.VehicleID AND r.ReturnDate IS NULL LEFT JOIN Customer c ON r.CustomerID = c.CustomerID ORDER BY v.VehicleNumber;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

VehicleNumber	VehicleModel	VehicleStatus	CustomerName	RentalDate	ReturnDate
ABC-123	Toyota Corolla	Available	NULL	NULL	NULL
ISB-321	Toyota Yaris	Available	Ayesha Noor	2026-09-11	NULL
KHI-654	Honda City	Available	Bilal Ahmed	2026-09-12	NULL
LEA-789	Suzuki Swift	Available	NULL	NULL	NULL
PSH-555	Toyota Fortuner	Available	NULL	NULL	NULL
RWP-987	Suzuki Wagon R	Maintenance	NULL	NULL	NULL
XYZ-456	Honda Civic	Available	NULL	NULL	NULL

The active-rental condition is placed in the JOIN so that non-rented vehicles remain visible.

5.4 Query 4

Display the total number of rentals made by each customer, including customers with no rentals.

- `SELECT c.CustomerID, c.CustomerName,
COUNT(r.RentalID) AS TotalRentals
FROM Customer c
LEFT JOIN Rental r ON c.CustomerID = r.CustomerID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalRentals DESC;`

Showing rows 0 - 6 (7 total, Query took 0.0005 seconds.)

```
SELECT c.CustomerID, c.CustomerName, COUNT(r.RentalID) AS TotalRentals FROM Customer c LEFT JOIN Rental r ON c.CustomerID = r.CustomerID GROUP BY c.CustomerID, c.CustomerName ORDER BY TotalRentals DESC;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

CustomerID	CustomerName	TotalRentals
1	Ali Khan	1
3	Hamza Raza	1
5	Bilal Ahmed	1
2	Sara Ahmed	1
4	Ayesha Noor	1
6	Fatima Ali	0
7	Usman Tariq	0

Activate Windows

COUNT(r.RentalID) gives zero for customers with no matching rental rows.

6. Task 4 — VIEW

- DROP VIEW IF EXISTS RentalReport;

```
CREATE VIEW RentalReport AS
SELECT
    r.RentalID,
    c.CustomerName,
    c.Phone,
    v.VehicleNumber,
    v.VehicleModel,
    r.RentalDate,
    r.ReturnDate,
    r.DailyRate,
    r.RentalCharge,
    p.PaymentAmount,
    p.PaymentDate,
    p.PaymentMethod
FROM Rental r
INNER JOIN Customer c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicle v ON r.VehicleID = v.VehicleID
LEFT JOIN Payment p ON r.RentalID = p.RentalID;

SELECT * FROM RentalReport;
```

```
CREATE VIEW RentalReport AS SELECT r.RentalID, c.CustomerName, c.Phone, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate, r.DailyRate, r.RentalCharge, p.PaymentAmount, p.PaymentDate, p.PaymentMethod FROM Rental r INNER JOIN Customer c ON r.CustomerID = c.CustomerID INNER JOIN Vehicle v ON r.VehicleID = v.VehicleID LEFT JOIN Payment p ON r.RentalID = p.RentalID;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, Edit, Copy and Delete features may result in undesired behavior. ⓘ

✓ Showing rows 0 - 4 (5 total, Query took 0.0011 seconds.)

```
-- Test VIEW SELECT * FROM RentalReport;
```

```
-- Test VIEW SELECT * FROM RentalReport;
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

		RentalID	CustomerName	Phone	VehicleNumber	VehicleModel	RentalDate	ReturnDate	DailyRate	RentalCharge	Payment
☐	Edit Copy Delete	1	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-01	2026-09-04	5000.00	15000.00	
☐	Edit Copy Delete	2	Sara Ahmed	0311-2345678	XYZ-456	Honda Civic	2026-09-05	2026-09-08	6500.00	19500.00	
☐	Edit Copy Delete	3	Hamza Raza	0322-3456789	LEA-789	Suzuki Swift	2026-09-10	2026-09-12	4000.00	8000.00	
☐	Edit Copy Delete	4	Ayesha Noor	0333-4567890	ISB-321	Toyota Yaris	2026-09-11	NULL	4500.00	0.00	
☐	Edit Copy Delete	5	Bilal Ahmed	0344-5678901	KHI-654	Honda City	2026-09-12	NULL	5500.00	0.00	

⬆ ☐ Check all With selected: Edit Copy Delete Export

The view provides one reusable consolidated rental report. It saves users from repeatedly writing the same multi-table JOINS and presents customer, vehicle, rental, and payment details together.

7. Task 5 — TRIGGER

The trigger prevents a vehicle from being inserted into a new active rental when that vehicle already has an active rental.

DROP TRIGGER IF EXISTS PreventDoubleBooking;

DELIMITER //

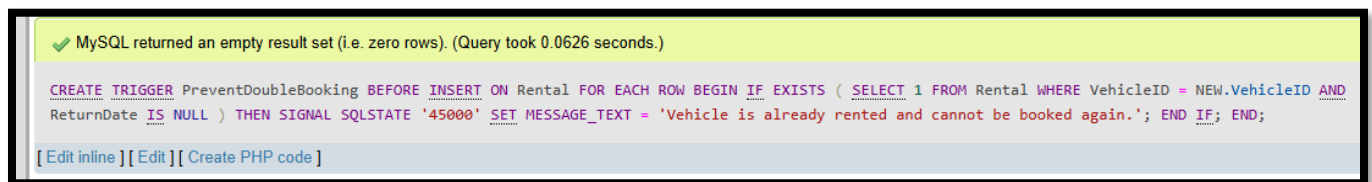
```
CREATE TRIGGER PreventDoubleBooking
BEFORE INSERT ON Rental
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Rental
```

```

WHERE VehicleID = NEW.VehicleID
AND ReturnDate IS NULL
) THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT =
'Vehicle is already rented and cannot be booked again.';
END IF;
END //

DELIMITER ;

```



MySQL returned an empty result set (i.e. zero rows). (Query took 0.0626 seconds.)

```

CREATE TRIGGER PreventDoubleBooking BEFORE INSERT ON Rental FOR EACH ROW BEGIN IF EXISTS ( SELECT 1 FROM Rental WHERE VehicleID = NEW.VehicleID AND
ReturnDate IS NULL ) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle is already rented and cannot be booked again.'; END IF; END;

```

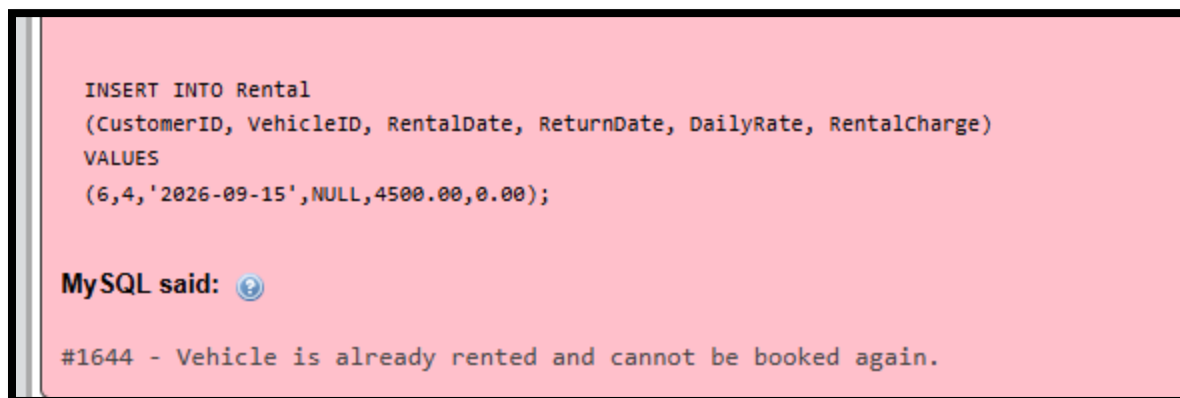
[Edit inline] [Edit] [Create PHP code]

7.1 Trigger Test

```

INSERT INTO Rental
(CustomerID, VehicleID, RentalDate, ReturnDate, DailyRate, RentalCharge)
VALUES
(6,4,'2026-09-15',NULL,4500.00,0.00);

```



```

INSERT INTO Rental
(CustomerID, VehicleID, RentalDate, ReturnDate, DailyRate, RentalCharge)
VALUES
(6,4,'2026-09-15',NULL,4500.00,0.00);

```

MySQL said: ?

#1644 - Vehicle is already rented and cannot be booked again.

Expected result: the INSERT is rejected with the message that the vehicle is already rented.

8. Task 6 — STORED PROCEDURE

```

DROP PROCEDURE IF EXISTS RegisterRental;

```

```

DELIMITER //

```

```

CREATE PROCEDURE RegisterRental(
IN p_CustomerID INT,
IN p_VehicleID INT,

```

```

    IN p_RentalDate DATE,
    IN p_ReturnDate DATE
)
BEGIN
    DECLARE v_DailyRate DECIMAL(10,2);
    DECLARE v_RentalDays INT;
    DECLARE v_RentalCharge DECIMAL(10,2);
    DECLARE v_ActiveRental INT DEFAULT 0;

    IF NOT EXISTS (
        SELECT 1 FROM Customer
        WHERE CustomerID = p_CustomerID
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Customer does not exist.';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM Vehicle
        WHERE VehicleID = p_VehicleID
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Vehicle does not exist.';
    END IF;

    SELECT COUNT(*)
    INTO v_ActiveRental
    FROM Rental
    WHERE VehicleID = p_VehicleID
    AND ReturnDate IS NULL;

    IF v_ActiveRental > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Vehicle is currently rented.';
    END IF;

    SELECT DailyRate
    INTO v_DailyRate
    FROM Vehicle
    WHERE VehicleID = p_VehicleID;

    IF p_ReturnDate < p_RentalDate THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =

```



```

        'Return date cannot be before rental date.';
    END IF;

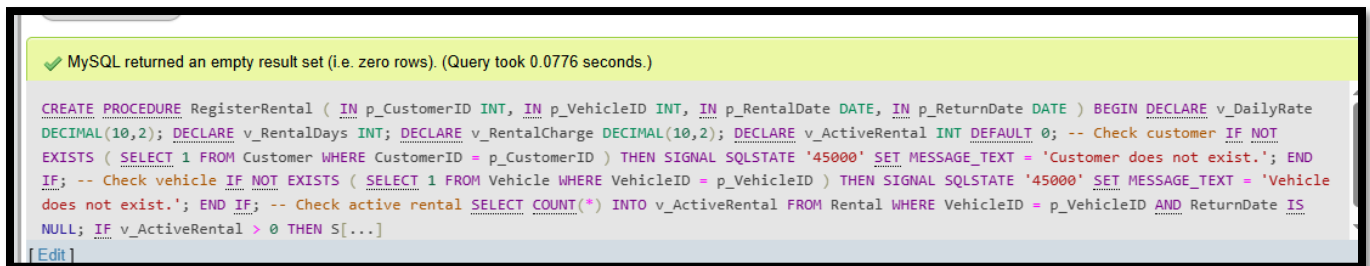
    SET v_RentalDays =
        GREATEST(DATEDIFF(p_ReturnDate, p_RentalDate), 1);

    SET v_RentalCharge =
        v_RentalDays * v_DailyRate;

    INSERT INTO Rental(
        CustomerID, VehicleID, RentalDate, ReturnDate,
        DailyRate, RentalCharge
    )
    VALUES(
        p_CustomerID, p_VehicleID, p_RentalDate, p_ReturnDate,
        v_DailyRate, v_RentalCharge
    );
END //

DELIMITER ;

```



MySQL returned an empty result set (i.e. zero rows). (Query took 0.0776 seconds.)

```

CREATE PROCEDURE RegisterRental ( IN p_CustomerID INT, IN p_VehicleID INT, IN p_RentalDate DATE, IN p_ReturnDate DATE ) BEGIN DECLARE v_DailyRate
DECIMAL(10,2); DECLARE v_RentalDays INT; DECLARE v_RentalCharge DECIMAL(10,2); DECLARE v_ActiveRental INT DEFAULT 0; -- Check customer IF NOT
EXISTS ( SELECT 1 FROM Customer WHERE CustomerID = p_CustomerID ) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer does not exist.'; END
IF; -- Check vehicle IF NOT EXISTS ( SELECT 1 FROM Vehicle WHERE VehicleID = p_VehicleID ) THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle
does not exist.'; END IF; -- Check active rental SELECT COUNT(*) INTO v_ActiveRental FROM Rental WHERE VehicleID = p_VehicleID AND ReturnDate IS
NULL; IF v_ActiveRental > 0 THEN S[...]
[Edit]

```

8.1 Procedure Test

```

CALL RegisterRental(
    6,
    7,
    '2026-09-15',
    '2026-09-18'
);

SELECT *
FROM Rental
ORDER BY RentalID DESC
LIMIT 1;

```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0585 seconds.)

```
CALL RegisterRental( 6, 7, '2026-09-15', '2026-09-18' );
```

[Edit inline](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ Showing rows 0 - 0 (1 total, Query took 0.0233 seconds.) [RentalID: 7... - 7...]

```
SELECT * FROM Rental ORDER BY RentalID DESC LIMIT 1;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

	RentalID	CustomerID	VehicleID	RentalDate	ReturnDate	DailyRate	RentalCharge
☐ Edit Copy Delete	7	6	7	2026-09-15	2026-09-18	12000.00	36000.00

[↑](#) ☐ Check all *With selected:* [Edit](#) [Copy](#) [Delete](#) [Export](#)

Vehicle 7 has a daily rate of 12,000. The procedure calculates 3 rental days and a total rental charge of 36,000.

9. Task 7 — Optimization Analysis

9.1 Identified Inefficiency

The system repeatedly checks Rental using VehicleID and ReturnDate to determine whether a vehicle is currently rented. As the Rental table grows, an unindexed search can examine many rows and reduce performance.

9.2 Improvement

```
CREATE INDEX idx_rental_vehicle_return
ON Rental(VehicleID, ReturnDate);
```

```
CREATE INDEX idx_rental_customer
ON Rental(CustomerID);
```

```
CREATE INDEX idx_rental_date
ON Rental(RentalDate);
```

The composite index supports the frequent active-rental lookup, while the other indexes support common customer and date-based searches. Indexes improve read performance but should not be added unnecessarily because they require storage and can add write overhead.

9.3 Improved Search Query

```
SELECT RentalID, CustomerID, VehicleID, RentalDate, ReturnDate
FROM Rental
```

WHERE VehicleID = 4
AND ReturnDate IS NULL;

```
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)
CREATE INDEX idx_rental_vehicle_return ON Rental(VehicleID, ReturnDate);
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0013 seconds.)
CREATE INDEX idx_rental_customer ON Rental(CustomerID);
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0013 seconds.)
CREATE INDEX idx_rental_date ON Rental(RentalDate);
[ Edit inline ] [ Edit ] [ Create PHP code ]

✓ Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)
-- Improved active-rental search SELECT RentalID, CustomerID, VehicleID, RentalDate, ReturnDate FROM Rental WHERE VehicleID = 4 AND ReturnDate IS NULL;
```

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	RentalID	CustomerID	VehicleID	RentalDate	ReturnDate
☐ Edit Copy Delete	4	4	4	2026-09-11	NULL

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

10. Management Reports

10.1 Customer Rental History

```
SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel,
       r.RentalDate, r.ReturnDate, r.RentalCharge
FROM Customer c
JOIN Rental r ON c.CustomerID = r.CustomerID
JOIN Vehicle v ON r.VehicleID = v.VehicleID
ORDER BY c.CustomerName, r.RentalDate;
```

Showing rows 1 - 9 (9 total, query took 0.0001 seconds) [CustomerName: ALI KHAN] [VehicleNumber: ABC-123] [RentalDate: 2026-09-01] [ReturnDate: 2026-09-04]

```
SELECT c.CustomerName, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate, r.RentalCharge FROM Customer c JOIN Rental r ON c.CustomerID = r.CustomerID JOIN Vehicle v ON r.VehicleID = v.VehicleID ORDER BY c.CustomerName, r.RentalDate;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate	RentalCharge
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04	15000.00
Ayesha Noor	ISB-321	Toyota Yaris	2026-09-11	NULL	0.00
Bilal Ahmed	KHI-654	Honda City	2026-09-12	NULL	0.00
Fatima Ali	PSH-555	Toyota Fortuner	2026-09-15	2026-09-18	36000.00
Fatima Ali	PSH-555	Toyota Fortuner	2026-09-15	2026-09-18	36000.00
Hamza Raza	LEA-789	Suzuki Swift	2026-09-10	2026-09-12	8000.00
Sara Ahmed	XYZ-456	Honda Civic	2026-09-05	2026-09-08	19500.00

10.2 Currently Rented Vehicles

```
SELECT v.VehicleNumber, v.VehicleModel,
       c.CustomerName, r.RentalDate
FROM Vehicle v
JOIN Rental r ON v.VehicleID = r.VehicleID
JOIN Customer c ON r.CustomerID = c.CustomerID
WHERE r.ReturnDate IS NULL;
```

```
SELECT v.VehicleNumber, v.VehicleModel, c.CustomerName, r.RentalDate FROM Vehicle v JOIN Rental r ON v.VehicleID = r.VehicleID JOIN Customer c ON r.CustomerID = c.CustomerID WHERE r.ReturnDate IS NULL;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

VehicleNumber	VehicleModel	CustomerName	RentalDate
ISB-321	Toyota Yaris	Ayesha Noor	2026-09-11
KHI-654	Honda City	Bilal Ahmed	2026-09-12

10.3 Vehicle Rental Activity

```
SELECT v.VehicleNumber, v.VehicleModel,
       COUNT(r.RentalID) AS TotalRentals,
       COALESCE(SUM(r.RentalCharge), 0) AS TotalRevenue
FROM Vehicle v
LEFT JOIN Rental r ON v.VehicleID = r.VehicleID
GROUP BY v.VehicleID, v.VehicleNumber, v.VehicleModel
ORDER BY TotalRentals DESC;
```

```
SELECT v.VehicleNumber, v.VehicleModel, COUNT(r.RentalID) AS TotalRentals, COALESCE(SUM(r.RentalCharge), 0) AS TotalRevenue FROM Vehicle v LEFT JOIN Rental r ON v.VehicleID = r.VehicleID GROUP BY v.VehicleID, v.VehicleNumber, v.VehicleModel ORDER BY TotalRentals DESC;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

VehicleNumber	VehicleModel	TotalRentals	TotalRevenue
PSH-555	Toyota Fortuner	2	72000.00
ISB-321	Toyota Yaris	1	0.00
KHI-654	Honda City	1	0.00
ABC-123	Toyota Corolla	1	15000.00
XYZ-456	Honda Civic	1	19500.00
LEA-789	Suzuki Swift	1	8000.00
RWP-987	Suzuki Wagon R	0	0.00

11. Final Verification

SELECT * FROM Customer;

```
SELECT * FROM Customer;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	CustomerID	CustomerName	Phone	Email	CNIC	City	Registration
☐ Edit Copy Delete	1	Ali Khan	0300-1234567	ali.khan@gmail.com	37405-1234567-1	Lahore	2026-09-19
☐ Edit Copy Delete	2	Sara Ahmed	0311-2345678	sara.ahmed@gmail.com	35202-2345678-2	Islamabad	2026-09-19
☐ Edit Copy Delete	3	Hamza Raza	0322-3456789	hamza.raza@gmail.com	61101-3456789-3	Karachi	2026-09-19
☐ Edit Copy Delete	4	Ayesha Noor	0333-4567890	ayesha.noor@gmail.com	37405-4567890-4	Lahore	2026-09-19
☐ Edit Copy Delete	5	Bilal Ahmed	0344-5678901	bilal.ahmed@gmail.com	35202-5678901-5	Rawalpindi	2026-09-19
☐ Edit Copy Delete	6	Fatima Ali	0355-6789012	fatima.ali@gmail.com	61101-6789012-6	Multan	2026-09-19
☐ Edit Copy Delete	7	Usman Tariq	0366-7890123	usman.tariq@gmail.com	37405-7890123-7	Peshawar	2026-09-19

SELECT * FROM Vehicle;

`SELECT * FROM Vehicle;`

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all |
 Number of rows:
 Filter rows:
 Sort by key:

Extra options

	VehicleID	VehicleNumber	VehicleModel	VehicleYear	DailyRate	VehicleStatus
☐ Edit Copy Delete	1	ABC-123	Toyota Corolla	2022	5000.00	Available
☐ Edit Copy Delete	2	XYZ-456	Honda Civic	2023	6500.00	Available
☐ Edit Copy Delete	3	LEA-789	Suzuki Swift	2021	4000.00	Available
☐ Edit Copy Delete	4	ISB-321	Toyota Yaris	2022	4500.00	Available
☐ Edit Copy Delete	5	KHI-654	Honda City	2023	5500.00	Available
☐ Edit Copy Delete	6	RWP-987	Suzuki Wagon R	2020	3500.00	Maintenance
☐ Edit Copy Delete	7	PSH-555	Toyota Fortuner	2024	12000.00	Available

☐ Check all
 With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

SELECT * FROM Rental;

`SELECT * FROM Rental;`

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all |
 Number of rows:
 Filter rows:
 Sort by key:

Extra options

	RentalID	CustomerID	VehicleID	RentalDate	ReturnDate	DailyRate	R
☐ Edit Copy Delete	1	1	1	2026-09-01	2026-09-04	5000.00	
☐ Edit Copy Delete	2	2	2	2026-09-05	2026-09-08	6500.00	
☐ Edit Copy Delete	3	3	3	2026-09-10	2026-09-12	4000.00	
☐ Edit Copy Delete	4	4	4	2026-09-11	NULL	4500.00	
☐ Edit Copy Delete	5	5	5	2026-09-12	NULL	5500.00	
☐ Edit Copy Delete	6	6	7	2026-09-15	2026-09-18	12000.00	
☐ Edit Copy Delete	7	6	7	2026-09-15	2026-09-18	12000.00	

SELECT * FROM Payment;

`SELECT * FROM Payment;`

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all |
 Number of rows:
 Filter rows:
 Sort by key:

Extra options

	PaymentID	RentalID	PaymentAmount	PaymentDate	PaymentMethod
☐ Edit Copy Delete	1	1	15000.00	2026-09-19	Cash
☐ Edit Copy Delete	2	2	19500.00	2026-09-19	Card
☐ Edit Copy Delete	3	3	8000.00	2026-09-19	Online

☐ Check all
 With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

SELECT * FROM RentalReport;

`SELECT * FROM RentalReport;`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	RentalID	CustomerName	Phone	VehicleNumber	VehicleModel	RentalDate
☐ Edit Copy Delete	1	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-01
☐ Edit Copy Delete	2	Sara Ahmed	0311-2345678	XYZ-456	Honda Civic	2026-09-05
☐ Edit Copy Delete	3	Hamza Raza	0322-3456789	LEA-789	Suzuki Swift	2026-09-10
☐ Edit Copy Delete	4	Ayesha Noor	0333-4567890	ISB-321	Toyota Yaris	2026-09-11
☐ Edit Copy Delete	5	Bilal Ahmed	0344-5678901	KHI-654	Honda City	2026-09-12
☐ Edit Copy Delete	6	Fatima Ali	0355-6789012	PSH-555	Toyota Fortuner	2026-09-15
☐ Edit Copy Delete	7	Fatima Ali	0355-6789012	PSH-555	Toyota Fortuner	2026-09-15

SHOW TRIGGERS;

`SHOW TRIGGERS;`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

Trigger	Event	Table	Statement	Timing	Created	sql_mode
			BEGIN			
PreventDoubleBooking	INSERT	rental	IF EXISTS (SELECT 1 ...	BEFORE	2026-09-19 06:19:42.13	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_

SHOW PROCEDURE STATUS

WHERE Db = 'cargo_rentals';

`SHOW PROCEDURE STATUS WHERE Db = 'cargo_rentals';`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Extra options

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set
cargo_rentals	RegisterRental	PROCEDURE	root@localhost	2026-09-19 06:21:29	2026-09-19 06:21:29	DEFINER		utf8mb4

Query results operations

12. Conclusion

The CarGo Rentals Management System was successfully designed and implemented as a relational database solution. The system separates customers, vehicles, rentals, and payments into normalized tables and uses primary keys, foreign keys, and integrity constraints to maintain reliable data. JOIN queries provide the required business reports, while the RentalReport view simplifies repeated reporting tasks. The double-booking trigger protects an important business rule, and the RegisterRental stored procedure performs validation and automatic rental-charge calculation. Appropriate indexes were also

proposed to improve frequently used rental searches. The resulting database provides a structured and practical solution for managing car rental operations.
